JULIA SHADERFORDÍTÓ

SZAKMAI GYAKORLAT BESZÁMOLÓ, 2025 NYÁR
SZŰCS GERGELY, P6ODRH

PROJEKTLEÍRÁS

A szakmai gyakorlat feladata egy olyan Julia könyvtár készítése volt, ami képes Julia-ban megírt kódot átfordítani GLSL shader kódba. Részét képezte még, hogy lehetőleg minél több minden általánosan legyen megírva a kódban, hogy később más célnyelvekre való fordítás (pl. HLSL) már könnyebben megcsinálható legyen, a teljes fordító újraírása nélkül.

A nyári szakmai gyakorlat idejére kizárólag a GLSL fordítás lett kitűzve célként, de a legeleje óta gondoltam a későbbi bővítésekre, így a kód alapját képző modulok teljesen újrahasználhatóak lesznek később újabb célnyelvek támogatása során.

A formai szempontból egy **Pkg** (a nyelv alapértelmezett csomagkezelője) csomagként lett megírva, egy bármilyen projektbe könnyen integrálható modulként. A fordító nyilvános API-ja többnyire egy függvényből áll, ami többek között a generált shader kódot adja vissza egy sima karakterláncként, így bármilyen környezetbe integrálható.

LEHETSÉGES ALKALMAZÁSOK

A projekt motivációját a **Juliagebra** projekt továbbfejlesztése adta, ami egy geometria vizualizációkat megjelenítő Julia szoftver. A hasonló alkalmazásokkal ellentétben, a megjelenítendő objektumok egy script-ként vannak definiálva, azaz Julia kóddal hozhatóak létre. Ezeknek a különböző számításokat végző függvényeit lehetne egyszerűen, dinamikusan párhuzamosítani a GPU oldalra való kihelyezéssel, ha lehetne belőlük shader kódot generálni. Azért is van erre szükség, mivel a tartalmuk előre nem ismert, a végfelhasználó kódján múlik, hogy éppen milyen számításokra van szükség.

Ehhez hasonlóan, bármilyen olyan projekthez jól alkalmazható a fordító, ahol a kívánt shader kód nem ismert fordítási időben, azaz szükséges valamilyen kódgenerálás a program futása alatt.

Egy másik lehetséges alkalmazás, shader-ek debug-olása. Ez általában egy nehéz feladatot jelent, mivel a GPU oldalon futó kódhoz nincsenek olyan komplexitású hibakereső eszközök, mint a CPU oldalihoz vannak. A Julia nyelven megírt shader-eken keresztül lehetőség nyílik a shader-ek logikájának CPU-oldali ellenőrzésére, mivel a Julia függvény (ami a shader alapját képezi) egy teljesen szokványos, Julia-ban lefordítható és futtatható függvény. Így akár hibakereséshez, de kicsit szélesebb körben teszteléshez is lehet őket felhasználni.

FEJLESZTÉSI IDŐVONAL ÉS SZEMÉLYES BESZÁMOLÓ

A fejlesztés július elején kezdődött, egy konzultációval, ahol megbeszéltük és kiosztottuk a projekteket, valamint felmértük, hogy mihez lesz szükség a munka megkezdéséhez.

Mivel nem volt még korábbi tapasztalom a Julia nyelvvel az első két héten a legtöbb időt dokumentáció olvasással töltöttem. Szerencsére a Julia egy jól dokumentált nyelv, az interneten és PDF-ben is elérhető dokumentáció tartalmaz a technikai részletek mellett egy kézikönyvet is, ami fejezetenként bemutatja a nyelv különböző koncepcióit. Ennek igyekeztem a fejlesztés megkezdéséhez szükséges tudást az első héten megszerezni, hogy a második héten már csak kiegészíteni kelljen azt, miközben elkezdődhet a fejlesztés is.

Először egy Shadertoy[1]-szerű alkalmazást fejlesztettem (a konzultáción megbeszéltek alapján), ami később egy hasznos eszközt nyújtott a fordító által generált shader-ek tesztelésére. Ez egy jó kiinduló pont volt, mivel a program maga nem tartalmazott nagyon logikai kihívást, így tudtam teljesen a nyelv használatának elmélyítésére koncentrálni. A projekt megtalálható a fordító mellett a feltöltött forráskódomban. Emellett azt a feladatot is kaptam a második hétre, hogy készítsek egy prezentációt a projekt megvalósítási tervével. Ennek során utánanéztem különböző nyelvek közötti fordító működésének (pl. Rust GPU[2]) és pár cikknek[3] a témában. Sikerült is összerakni egy olyan architektúrát, amit utána lehetett a projekt végéig követni, nagyobb módosítások nélkül (ennek részleteit később ismertetem).

A következő 4 hétben próbáltam egy minimális shader fordításához szükséges funkcionalitást megírni. Ezalatt hétről hétre látszott, ahogy halad előre a fordítási procedúra, de mindig csak köztes részeredményeket láttam, ami valamennyi bizonytalansággal töltött el, mivel ez volt az első ilyen komplexitású nyelvi fordítással foglalkozó projekt, amin dolgoztam. Szerencsére a 6. héten sikerült egy Hello World típusú shader-t lefordítani a csomaggal, innen már magabiztosan tudtam folytatni a további fejlesztéseket. Ezalatt a pár hét során fejlesztettem egy segédcsomagot is, ami a különböző fastruktúrák dinamikus logika alapján való bejárásához nyújt egy könnyen használható felületet. Ez szintén megtalálható a leadott forráskódomban. Ezen keresztül sikerült megtanulnom azt is, hogyan működik a Julia beépített csomagkezelője, ami a fordítóhoz is később jól jött. Továbbá megismerkedtem a Julia absztrakt szintaxisfájának (AST) struktúrájával, feldolgozásával és az azon keresztül elérhető metaprogramozási lehetőségekkel is, ami az egész fordító alapját képezi.

---

[1] https://www.shadertoy.com/
[2] https://rust-gpu.github.io/
[3] Például a leghasznosabb: https://tomassetti.me/how-to-write-a-transpiler/

A maradék időben a fordító továbbfejlesztésével foglalkoztam, extra nyelvi konstrukciók támogatásával, meglévő funkciók kibővítésével és javításaival, demó shader kódok összerakásával és minél több GLSL funkció implementálásával. Itt foglalkoztam fordítóprogramok elméletével kapcsolatos problémákkal, Julia-oldali optimalizációkkal és OpenGL-specifikus funkciókkal is. Továbbá írtam függvény implementációkat a **JuliaGLM** segédkönyvtárhoz is, hogy a shader-ek Julia-oldali futtatása is lehetséges legyen a jövőben.

PROJEKT STRUKTÚRA ÉS FORDÍTÁSI FOLYAMAT

A projekt a nyelvi fordítást több egymás utáni fabejárás és transzformáció sorozataként végzi el. Minden ilyen transzformációs lépést a fordítóban *stage*-nek kereszteltem el, és ezeknek sorozatát hívtam *pipeline*-nak. A pipeline-ok felelősek a tényleges fordítás elvégzéséért, míg a *stage*-ek egy-egy konkrét feladatot végeznek el azon belül.

A stage-ek többnyire csoportosíthatóak aszerint, hogy általános Julia kód feldolgozási szerepet töltenek be, vagy a célnyelv-re való fordításhoz szükséges transzformációt végeznek-e. A nyelv-független fordítási lépések együtt alkotják az általános részét a fordítónak, amit akár később más célnyelvekre való fordításhoz is fel lehet használni. Ez a következő 4 stage-ből áll:

1. Preprocesszor
2. Scope-ok hatókörének és típusának meghatározása
3. Szimbólum rezolúció
4. Típuskövetkeztetés

Ez a pár lépés után kiegészíthető akár eléjük, közéjük vagy utánuk helyezett további fordítási lépésekkel. A GLSL fordító például a következő extra lépésekkel egészíti ezt ki:

1. GLSL preprocesszor
2. *A fenti 4 lépéses procedúra (2-5. lépés)*
6. IR transzformáció
7. Kódgenerálás

A lépések konkrét feladatát a következő szekcióban fogom ismertetni, először még bemutatom a fordítás végrehajtásának pontos folyamatát és hogy ezek a lépések pontosan hogyan is működnek együtt.

A fordítás egy pipeline lefuttatását jelenti valamilyen Julia kóddal, ami pedig egy stage-sorozat lejátszását végzi el a bemeneti kód AST-jén. Először a fordító inicializál egy pipeline kontextust, aminek típusát a pipeline szabja meg. Ez egy felületet nyújt arra, hogy a fordítási lépések egymás között is tudjanak kommunikálni, valamint arra, hogy a fordító általános részei tudjanak célnyelv-függő logikát is figyelembe venni a kontextuson keresztül (pl. beépített változók és függvények típusának kezelése).

A kontextus létrehozása után a fordító elkezdi a pipeline stage-eit lefuttatni (igazából először itt még lefordítja az esetleges segédfüggvényeket, de ezt külön pontban fogom kifejteni). Az általános szabály a stage-ek futása során, hogy az első stage-nek bemenetként el kell fogadnia a kapott kód AST reprezentációját, majd minden későbbi stage-nek a bemenete az előző kimenete lesz. Azaz, például a típuskövetkeztetés utáni stage egy *TypedAST* típust kell várjon bemenetként, különben nem fog lefutni a pipeline.

A stage-ek ezen a „láncolt” bemeneti struktúrán kívül még megkapják a pipeline kontextust és a fordítást kezdeményező Julia modult is bemenetként.

A sikeres fordítás eredménye a generált kód String-ként és egy Julia függvény definíció, ami futtatható, hogy megkapjuk a shader-el (többnyire) ekvivalens Julia függvényt. Igazából a kód visszaadásánál nem köti meg a fordító, hogy String-ként történjen, mivel így később akár olyan pipeline-ok is létrehozhatók, amik utolsó lépésként akár rögtön a projektben futó OpenGL környezeten keresztül összerakják automatikusan a program objektumot, lefordítják OpenGL-el a shader-t és a program azonosítóját adják vissza. Ilyen szempontból a pipeline-ok integrációs eszközként is felhasználhatóak.

ÁLTALÁNOS STAGE-EK

PREPROCESSZOR

A preprocesszor feladata a bemenetként kapott AST előkészítése a későbbi stage-ek számára. A kimenete ugyanúgy egy Julia AST, ami már könnyebben feldolgozható a fordító által. Itt felold a fordító néhány könnyen elkerülhető „syntactic sugar”-t, mint például a többszörös láncolt összehasonlítások: az `(a < b < c)` kifejezés például könnyen lecserélhető `(a < b && b < c)`-re. Ha már magát a Julia kódot cseréljük le, akkor a későbbi lépésekben eggyel kevesebb nyelvi konstrukciót kell figyelembe venni, anélkül, hogy szemantikailag változtatnánk a kód jelentését. Több ilyen ekvivalens átalakítás is szerepel a preprocesszorban, bár néhánynál szigorításokat kell tenni arra, hogy milyen esetekben végezhető el az átírás. Például `(a, b = 1, 2)` ugyanazt jelenti, mint `(a = 1 ; b = 2)`, viszont `(a, b = b, a)` már nem oldható fel hasonló módon.

Emellett a preprocesszor még a lebegőpontos számliterálok közötti inkonzisztenciák feloldására is szolgál a két nyelv között. GLSL-ben a `2.0` típusú literálok egy 32-bites lebegőpontos számot jelölnek, míg Julia-ban ugyanez egy 64-bites literál. A `Float32`-höz tartozó literálokat pedig normálalakban kell megadni, itt például `2.0f0` lenne. Az egységesség érdekében, a fordító az első lépés során átalakítja az összes 64-bites literált a 32-bites megfelelőjére. Így közelebb áll egymáshoz a Julia-ban megírt és a generált shader kód, valamint Julia oldalon is jelentősen kevesebbet kell írni, ha nem kell mindenhova kiírni az `f0` utótagot. Ez a funkció természetesen kikerülhető, ha a literált egy `Float64` konstruktorába írjuk (ekkor GLSL oldalon `double` fog képződni), vagy fordító szinten is kikapcsolható a `transpiler_config.literals_as_f32` átállításával.

```
function preprocessor_test()
    x, y, z = 1, 2, 3

    if x < y < z
        x += y
    end
end
```

```
function preprocessor_test()
    x = 1
    y = 2
    z = 3

    if x < y && y < z
        x = x + y
    end
end
```

Példa egy feldolgozott kódra

SCOPE-OK FELDERÍTÉSE

A következő lépés a scope-ok pontos behatárolása. Ez főleg csak egy előkészítő lépése a következő stage-nek, amikor megállapítja a fordító, hogy melyik szimbólum mire mutat pontosan. Ez magában nem egy túl komplex feladat, a Julia dokumentációban található egy lista, ami pontosan bemutatja, hogy melyik nyelvi konstrukciók hoznak be új scope-ot (és azok milyen típusúak). Julia-ban kétféle scope típus létezik: soft és hard. A két típusban más szabályok szerint kerül eldöntésre, hogy egy adott szimbólum éppen mire mutat a scope-ban. Azonban ahogy a dokumentációból[4] levezethető, mivel minden esetben egy fordított környezetben futtatott függvény definícióban szerepel a kód (ami egy hard scope), így ez egy fix értelmezést ad minden szimbólumnak, a gyerek scope-ok típusától függetlenül. Ettől függetlenül a lépés meghatározza a típust is, hátha későbbi továbbfejlesztés során hasznos információ lesz.

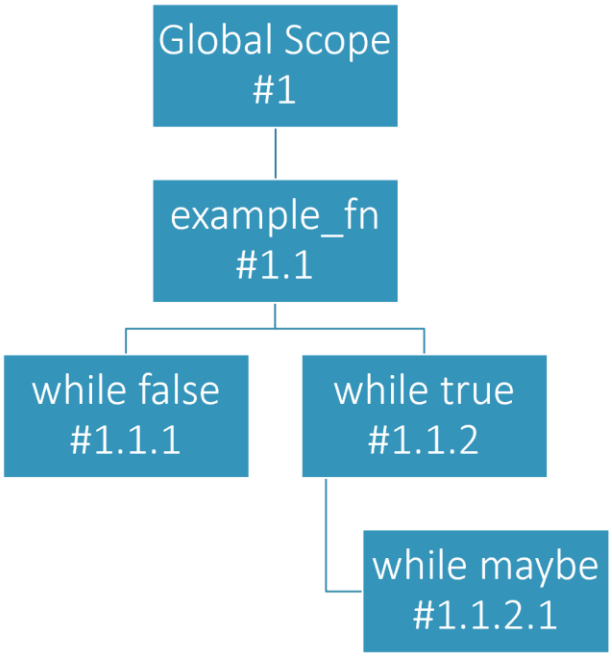
A scope-ok azonosítására egy egyszerű hierarchikus azonosító rendszert használ a stage:

1. A globális scope azonosítója #1
2. A többi scope azonosítója a szülő azonosítója, hozzáfűzve a gyerek indexét

A globális (#1) scope után a függvény definíció scope-ja következik, ami az #1.1-es azonosítót fogja kapni mindig. Ezen belül az első scope-ot nyitó konstrukció az #1.1.1-es azonosítót kapja, a második az #1.1.2-eset, és így tovább. A következő ábra szemlélteti ezt a számozást:

[4] https://docs.julialang.org/en/v1/manual/variables-and-scoping/

```julia
# Scope 1

function example_fn()
    # Scope 1.1

    while false
        # Scope 1.1.1
    end

    while true
        # Scope 1.1.2

        while rand() < 0.5
            # Scope 1.1.2.1
        end
    end
end
```



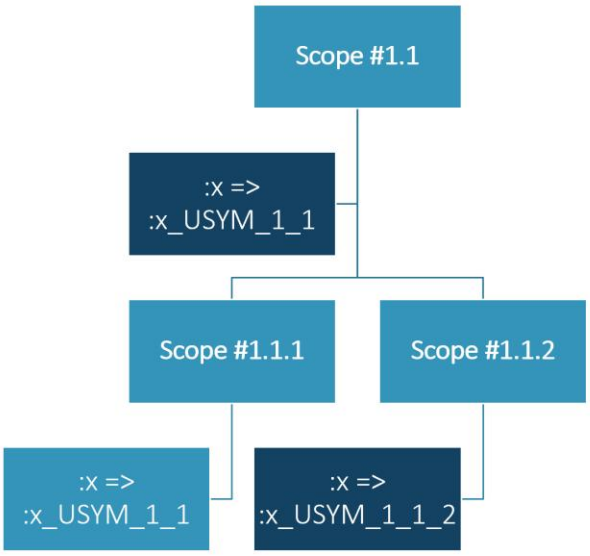
SZIMBÓLUM REZOLÚCIÓ

Ebben a stage-ben a fordító meghatározza minden szimbólum használatról, hogy az adott név hova mutat pontosan. Ez azért fontos, mert így később kódgenerálás során lehet előre deklarálni az összes változót, és nem kell aggódni a Julia és a célnyelv közötti scope szabályok eltérésein. Ez azért is hasznos, mivel Julia-ban a deklarációk, scope-ok és szimbólum meghatározások elég engedékenyen vannak kezelve.
A legtöbb modern GLSL fordító elméletileg rendelkezik olyan funkciókkal, hogy ennek nem kéne problémát okoznia optimalizáció szempontjából, de ettől függetlenül a jövőben egy szebb megoldás lenne, ha a fordító figyelembe venné, hogy mi az ideális hely a kódban az egyes deklarációk elhelyezésére.

A szimbólum rezolúció maga végigmegy sorban a scope-okon és felderíti azokban az egyes szimbólumokhoz köthető felhasználásokat. Ezek lehetnek különböző típusúak (deklaráció, értékadás, értéklekérés), amik sorrendje és jelenléte mind külön-külön befolyásolja, hogy az adott szimbólum hova mutat. A stage minden egyedi változóhoz (azaz adott nevű, adott scope-ban definiált változó) hozzárendel egy egyedi címkét, az definiáló scope azonosítójával együtt (pl. x a függvény scope-jában x_USYM_1_1 lesz).

Ezután a szimbólum használatokat kicseréli a stage arra az egyedi azonosítóra, amire mutatnak.

```julia
function usym_example()
    some_global = 2

    global some_global

    some_other_global = 2

    x = 1

    while false
        x = 2
    end

    while false
        local x
        x = 3
    end

    some_global = 3
end
```



Az *x* szimbólum különböző előfordulásait bemutató fa

```
#version 330 core

// This code was auto-generated by the Julia -> GLSL transpiler

void main() {

    // USYM DECLARATIONS

    int some_other_global_USYM_1_1;
    int x_USYM_1_1_2;
    int x_USYM_1_1;

    // BODY

    some_global_USYM_1 = 2;
    some_other_global_USYM_1_1 = 2;
    x_USYM_1_1 = 1;
    while (false) {
        x_USYM_1_1 = 2;
    }
    while (false) {
        x_USYM_1_1_2 = 3;
    }
    some_global_USYM_1 = 3;
}
```

A fenti kód GLSL-be fordított alakja
(egy hiba miatt itt hiányzik a some_global uniform-ként történő deklarációja)

TÍPUSKÖVETKEZTETÉS

Ez az utolsó lépése az általános fordításnak. A fordító itt elindul az AST levélelemein, amiknek ismert a típusa (pl. literálok, paraméterek, globális változók), majd innen elindul felfele és meghatározza minden belső elem típusát is. Talán ez a fordító legkomplexebb része, mivel minden nyelvi konstrukció a saját típuskövetkeztető szabályaival rendelkezik. Például egy indexelés típusa más típust fog visszaadni attól függően, hogy vektorba vagy mátrixba történik az indexelés, de attól függően is, hogy milyen fajta index van alkalmazva (mátrixokból lekérhetők oszlopvektorok és konkrét elemek is). A JuliaGLM könyvtár által támogatott swizzle lehetőségek közül a fordító mindegyiket támogatja, azok eredmény típusát itt kell eldönteni.

Azt is számon kell tartani, hogy az egyes egyedi szimbólumok milyen típusokat kapnak értékadásokon keresztül. Mivel a Julia egy dinamikus típusrendszerrel rendelkező nyelv, itt lép elő az a szigorítás, hogy nem használhatók shader kódban olyan változók, amiknek a típusa újradefiniálásra kerül. Ennek részleges támogatása lehetséges lenne különböző generált változók létrehozásával, de komplex logika kéne a feltételes és ciklusos konstrukciók ellenőrzéséhez (pl. ha egy feltételes blokkban kerül megváltoztatásra a típus, a fordítónak nem eldönthető onnantól, hogy melyik típust kéne használnia).

A leggyakoribb elemek az AST-ben a függvényhívások. Mivel a fabejárás a levélelemeknél kezdődik, így amikor a fordítónak el kell döntenie, hogy egy függvényhívás milyen típusú értéket ad vissza, már ismeri a paraméterek típusát. Ezen keresztül le lehet kérdezni Julia-ban a visszatérési értéket, bár ez nem mindig nyújt megoldást. A 32-bites float-ok összeadása például a típushierarchiában legfelül található Any típust adja vissza, mivel néhány esetben megtörténhet, hogy a Julia egy 64-bites float-ot ad vissza a precizitás megőrzése végett. GLSL-ben persze nincs ilyen, ott két float összeadása mindig float-tal fog visszatérni. Az ilyen edge case-eket figyelembe kellett venni itt.

Emellett kellett egy felületet nyújtani arra is, hogy az egyes pipeline-ok megadhassák, hogy az ő célnyelvük milyen beépített függvényekkel rendelkezik, és azoknak mi a típusszignatúrája. Ezt a kontextusukon keresztül tehetik meg az egyes pipeline-ok, amit a típuskövetkeztetés fel tud használni a függvényhívások típusának meghatározására.

```
x = 1.0
y = 2.0

z = x + y
```

```
[ASTFloat32]
Expr: =
  [ASTVoidSym]
  Symbol: z_USYM_1_1
  [ASTFloat32]
  Expr: call
    [ASTFunction]
    Symbol: +
    [ASTFloat32]
    Symbol: x_USYM_1_1
    [ASTFloat32]
    Symbol: y_USYM_1_1
```

GLSL PIPELINE

Ezen felül a GLSL pipeline rendelkezik még három saját stage-el, de ezeken nem mennék végig ilyen részletesen, mivel többnyire egy-két mondattal megfogható mit csinálnak.

A GLSL preprocesszor azért felelős, hogy mielőtt az általános pipeline megkapja a kódot, kinyerjen belőle minden GLSL-specifikus extra információt. Ilyenek pl. a változókhoz tartozó qualifier-ek (pl. flat, const, stb.).

Az IR transzformációs stage azért felelős, hogy a típusokkal rendelkező AST-t egy GLSL-hez közelebbi formátumba hozza, ahonnan már könnyebb elvégezni a kódgenerálást. Emellett végez pár GLSL-specifikus ellenőrzést is a kódon (pl. swizzle-ek helyessége).

A kódgenerálás végül pedig csak az előző stage-ből kapott IR-ből egy GLSL shader kódot tartalmazó String-et generál, ami a visszatérési értéke az egész pipeline-nak is.

```
julia> m = mat3(1:9)
3×3 Matrix{Float32}:
 1.0  4.0  7.0
 2.0  5.0  8.0
 3.0  6.0  9.0
```

```
julia> m[3,2]
6.0f0
```

```
julia> m[6]
6.0f0
```

```
julia> m[:,2]
3-element StaticArr
 4.0
 5.0
 6.0
```

```
@exported struct GLSLMatIndexer <: GLSLASTNode
    target::GLSLASTNode
    column::GLSLASTNode
    row::Union{GLSLASTNode,Nothing}
end
```

```
float six = m[1][2];
```

```
vec3 second_col = m[1];
```

Példa a mátrixindexelés feldolgozására, ami arra is rámutat, hogy miért hasznos egy köztes állapotra átváltani kódgenerálás előtt

SEGÉDFÜGGVÉNYEK

A fordító támogat több függvényt definiáló shader-eket is. Ezeket a segédfüggvényeket a fő shader függvényen belül, annak legelején kell definiálni. Mielőtt a fordító megkezdi a main függvény fordítását, fel fogja dolgozni az összes ilyen segédfüggvényt és azokból önálló GLSL kódot generál.

Az ezekből generált kód, valamint a függvény típusszignatúrája a pipeline kontextusába kerül, ahonnan pl. a típuskövetkeztetés során is elérhető, feltípusozható általa a segédfüggvények meghívása is. Végül a kódgenerálás során ezek definíciója bekerül a generált kód tetejére, hogy a visszaadott String tartalmazzon mindent, ami fordításhoz szükséges.

Egy limitációja ennek a rendszernek, hogy a segédfüggvények között tett hívásokat nem deríti fel előre a fordító, így a felhasználó felelőssége, hogy a segédfüggvények használati sorrendben kerüljenek definiálásra (és így fordításra is).

LEHETSÉGES TOVÁBBFEJLESZTÉSEK

A fordító jelenlegi állapota inkább egy jól működő prototípus, mint egy használatra kész könyvtár. Sajnos a nyári szakmai gyakorlat idején nem volt már idő ennél fejlettebb állapotra hozni a projektet, viszont a jövőben a következő lista szerint lehetne javítani és kibővíteni a fordító működését:

1. Bővebb GLSL támogatás
    a. Tömbök
    b. Struct-ok
    c. Interface blokkok
    d. Hiányzó GLM és GLSL függvények pótlása
2. Julia-oldali optimalizációk
    a. Jelenleg a projekt a legelső fordítás során lassabban fog teljesíteni, mint későbbi végrehajtások során, a Julia JIT fordítási modellje miatt.
    Ez már egy javított állapot a kezdeti teljesítményhez vetve, de még van hova javítani ezt
3. Jobb hibakezelés
    a. A fordító jelenleg nyújthatna segítőkészebb hibaüzeneteket, részletes információkkal (pl. hibás kódrészlet fájlja és sorszáma)
4. Előre deklarált változók újragondolása
    a. Ez részletezésre került a Szimbólum rezolúció szekcióban